

Subject: CS336, Spring, 2026

Date: from July 20, 2026 to August 13, 2026

Contents

Lecture 2: PyTorch (einops) & Memory Accounting

Reference: CS336, Lecture 2: PyTorch (einops)

Introduction

在 Lecture 1 中提到本课程的一个重要的问题是：“给定计算资源，如何训练出最好的模型”，或者说 “How to maximize computational efficiency?” 在此之前，我们需要先介绍计算资源 (computational resources) 是什么。

Mindset: 资源计算 (resource accounting)，对可训练模型的大小上限和需要时间与硬件资源进行大致估算。Resource accounting 主要包括 ① 计算量 (Compute) ② 存储量 (Memory) ③ 数据移动 (Memory bandwidth)。

Basic Knowledge

Tensor Memory

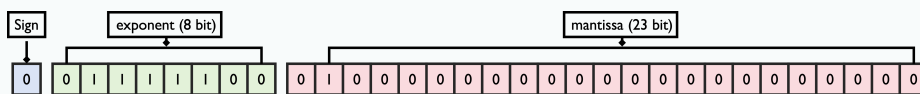
Tensor 是 PyTorch 等深度学习框架中存储数据和进行计算的基本单位，其可以存储例如 parameter, gradient, data, optimizer state 等一切基本内容。Tensor memory 由两部分决定：① 元素数量 ② 每个元素的数据类型(dtype)。

Binary: 在十进制中浮点数的表示可以写为 $m \times 10^n$ ，例如 $6500 = 6.5 \times 10^3$ 。而二进制 (binary) 中浮点数一般表示为 $x = (-1)^s \times 2^e \times m$ ，其中 ① s 为符号位 (sign) ② e 为指数位 (exponent) ③ m 为尾数位 (mantissa)。转换公式为：

$$x = (-1)^s \times 2^{e-\text{bias}} \times \left(1 + \frac{F}{2^m}\right), \quad (1)$$

其中 bias 为指数偏移量， F 为尾数字段按无符号整数读取后的值。此公式适用于正规数 (normal number)。

Float 32. 也被称为 IEEE 754 single precision，是深度学习中的默认数据类型。float 32 使用 32 bits = 4 bytes 表示一个浮点数，其中符号位 1 bit，指数位 8 bit，尾数 23 bit。



根据公式 (1)，图中所示的 float 32 表示的浮点数转换为 10 进制应该为：

$$(-1)^0 \times 2^{1 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 + 1 \times 2^5 + 1 \times 2^6 - 127} \times \left(1 + \frac{1 \times 2^{21}}{2^{23}}\right) = 1 \times 2^{124-127} \times 1.25 = 1 \times 2^{-3} \times 1.25 = 0.15625.$$

Float 16. 每个元素占 16 bits = 2 bytes，其中符号位 1 bit，指数位 5 bit，尾数 10 bit。

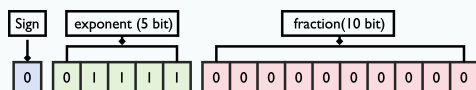


Figure 1: Float 16

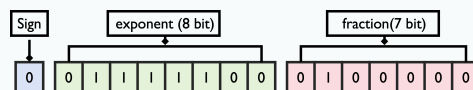


Figure 2: BFloat 16

BFloat16. 即 brain floating point，每个元素占 16 bits = 2 bytes，其中符号位 1 bit，指数位 8 bit，尾数 7 bit。

Note 1. Float 16 与 BFloat 16 相较于 Float 32 都减半了存储量，但 Float 16 的尾数位更多，因此精度更高 (可以表示更精细的小数)，而 BFloat 16 的指数位更多 (与 Float 32 相同)，因此数值范围更大 (可以表示绝对值更大的数)。

Float 8. 每个元素只有 8 bits = 1 byte，分为 FP8 E4M3 和 FP8 E5M2 两种。FP8 E4M3 符号位 1 bit，指数位 4 bit，尾数 3 bit，FP8 E5M2 符号位 1 bit，指数位 5 bit，尾数 2 bit。因此 E4M3 精度更高，E5M2 数值范围更大。

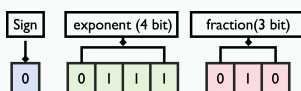


Figure 3: FP8 E4M3

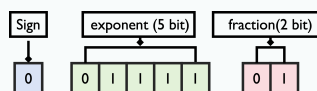


Figure 4: FP8 E5M2

Mixed precision training. 在 LLM 的整个训练 pipeline 中各个部分 (如前向传播、反向传播、优化器等) 所需要的最低精度可能不同，因此可以使用混合精度，例如使用 float 32 用于计算 attention，使用 bfloat 16 计算前向传播。

Tensor Einops

Reference: Einops tutorial

Tensor

Tensor 是 DL 中重要的数据类型，是具有统一 dtype 的多维数组。PyTorch 中 tensor 是底层存储的一种视图，可简单理解为对已分配内存 (storage) 的指针 (pointer)。Tensor 的底层 storage 可以看成一条长数组，如图 5 6 所示。

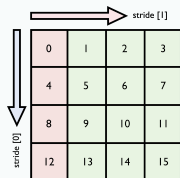


Figure 5: 4×4 tensor, `tensor.stride[0] = 4`

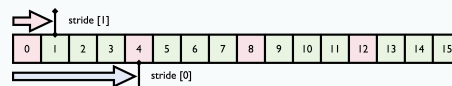


Figure 6: Tensor in different directions

在索引 tensor 中元素 (x, y) 时，只需要在图 6 所示的长数组中找到第 N 个元素即可，索引计算方式为^a:

$$N = x \times \text{stride}[1] + y \times \text{stride}[0]. \quad (2)$$

① PyTorch 中很多对于 tensor 的操作实际上只是在创建新的视图 (view) 而无需重新分配内存，这被称为切片 (slice)，详见 [Slicing, Indexing, and Masking](#)。② 大部分对 tensor 的操作默认其是连续的 (contiguous)，即按照图 6 的方式按顺序连续索引，但例如转置 (`tensor.t()`) 后的 tensor 和原 tensor 共享内存导致在图 6 中跳跃遍历 (内存地址可能发生跳跃)，会返回不连续 (non-contiguous) 提示。此时可以使用 `x.contiguous()` 重新分配内存。

^a注意这里 tensor 和数组中索引都是从 0 开始。

Tensor Einops

Einops 是用于操作 tensor 形状和维度的工具库，其核心思想是不再用一连串 `reshape`、`transpose`、`permute` 表达操作，而是直接写出输入和输出各个轴的含义，因此灵活性和可读性更高。

普通 PyTorch 操作需要明确知晓并对齐维度，但 Einops 使用明确的描述进行操作。

```
1 import torch
2 from einops import einsum
```

transpose: 例如在进行 tensor 乘法时需要将维度对齐，常规需要使用 `transpose` 对齐且需要记住维度对应含义。

```
1 B = 2 # batch size
2 T1 = 4 # sequence length
3 T2 = 5 # sequence length
4 D = 8 # hidden dimension
5 x = torch.ones(B, T1, D)
6 y = torch.ones(B, T2, D)
7
8 z1 = x @ y.transpose(-2, -1) # torch.Size([B, T, T]) = torch.Size([2, 4, 4])
9 z2 = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
10
11 assert z1.shape == z2.shape
```

rearrange: 例如在 Transformer 中多头注意力 (B, T, D) 有时需要进行拆解为 (B, T, H, DH) 需要使用 `rearrange`。

```
1 H = 2 # head
2 Dh = 4 # hidden dimension per head
3 m1 = x.reshape(B, T1, H, Dh).permute(0, 2, 1, 3) # B, H, T, Dh
4 m2 = rearrange(x, "b t (heads dh) -> b heads t dh", heads=H)
5 assert m1.shape == m2.shape
```

reduce 使用 Einops 可以直接明确标注需要操作的名称和位置，例如求和、求均值等。

```
1 x = torch.ones(2, 4, 2)
2 n1 = x.sum(dim=-1)
3 n2 = reduce(x, "b t d -> b t", "sum")
4
5 n3 = x.mean(dim=-1)
6 n4 = reduce(x, "b t d -> b t", "mean")
```

Tensor Operations FLOps

Tensor Operations FLOps

FLOP: 浮点数运算 (floating-point operation, FLOP) 是指一次基本运算, 包括加法 ($x + y$) 和乘法 ($x \times y$).

FLOP/s: 每秒能够完成的浮点数运算量 (floating-point operation per second, FLOP/s), 有的地方会写为 FLOPS.

Note 2. *FLOPs* 用于衡量计算量, 而 *FLOP/s* 用于衡量硬件的计算速度. 例如一操作需要 10^{12} *FLOPs*, 而 *GPU* 实际吞吐量是 10^{14} *FLOP/s*, 那么只考虑计算的理想时间是: $\frac{10^{12}}{10^{14}} = 0.01$ 秒, 不过实际时间通常更长.

Example 1. ① *Matmul*: 对于矩阵乘法 (*matrix multiplication, matmul*) $Y = XW$, $X \in \mathbb{R}^{B \times D}$, $W \in \mathbb{R}^{D \times K}$, 其需要 $2BDK$ *FLOPs*, 其中乘法 BDK *FLOPs*, 加法 BDK *FLOPs*. ② 对于矩阵加法 $A + B$, $A, B \in \mathbb{R}^{m \times n}$, 需要 mn *FLOPs*.

FLOP 记录的是所需要的计算量, 实际所需要的时间还要取决于硬件的计算速度, 但需要注意的是硬件的计算速度一般标注的都是峰值计算速度, 一般计算场景中无法达到. 例如 NVIDIA RTX PRO 6000 Blackwell Workstation Edition 标注的 Single-Precision Performance (peak rates) 为 126 TFLOP/s.

Model FLOPs Utilization (MFU)

MFU. 虽然 GPU 会标写理论峰值算力, 但实际计算中由于多种原因模型的训练/推理并不能完全利用这些算力, 而模型实际使用算力与理论峰值算力的比值就被定义为模型浮点运算利用率 (Model FLOPs Utilization, MFU):

$$MFU = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}. \quad (3)$$

Note 3. ① 高 MFU 意味着 GPU Tensor Core 使用得更充分、单位时间处理的 *token* 更多. ② MFU 与 GPU Utilization 不一样, 在 `nvidia-smi` 中看到的 *GPU utilization* 通常近似表示在采样时间内 GPU 有多少 *kernel* 正在运行, 但不能说明 *kernel* 是否充分使用了 Tensor Core 进行计算. ③ 理论峰值通常只在非常理想的大型矩阵乘法中才能接近.

Note 4. *FLOP/s* 取决于两点: ① GPU ② 浮点数类型, 例如一般 $BFloat16 \gg Float32$.

Arithmetic Intensity

在实际训练中数据在 GPU 中需要不断在 memory 与 accelerator 间移动: ① 输入从 memory 移动至 accelerator → ② 进行计算 → ③ 输出从 accelerator 移动至 memory. 衡量数据移动速度的量为内存带宽 (memory bandwidth) (bytes/s), 衡量 accelerator 计算速度的量即 FLOP/s, 二者比值称为算术强度 (arithmetic intensity):

$$\text{Arithmetic Intensity} = I = \frac{\text{FLOPs}}{\text{Bytes Moved}}. \quad (4)$$

Arithmetic intensity 衡量了每从显存搬运 1 byte 数据能够完成的浮点运算次数. Arithmetic intensity 较低说明搬运很多数据但只需少量计算, 较低则反之. 考虑到具体训练场景, 如果 communication time ($\frac{\text{bytes}}{\text{bytes/s}}$) 小于 computation time ($\frac{\text{FLOPs}}{\text{FLOP/s}}$) 那么 memory 对数据的提供速度超过计算单元的消耗速度, 模型训练受到 compute 制约, 因此称为 **compute-bound**, 反之模型训练受到 memory 制约, 称为 **memory-bound**.

Note 5. Arithmetic intensity 针对的是某个计算任务, 其可以是一个简单的 tensor operation 也可以是整个训练过程.

考虑 BFloat16 的浮点数类型 (16bits = 2 bytes), 并使用 H100 GPU, 下面给出几种计算任务的 arithmetic intensity 计算过程.

Example 2 (ReLU). 对于 ReLU 激活函数 $y_i = \max(0, x_i)$ ($x \in \mathbb{R}^N, N = 1024^2$), 读取 x 和写入 y 均需要 $2N$ bytes, 计算需要 N FLOPs, 因此 $I_{\text{ReLU}} = \frac{N}{4N} = 0.25 \text{ FLOPs/byte}$. 此时 communication time ($1.252 \times 10^{-6} \text{ s}$) $>$ computation time ($1.060 \times 10^{-9} \text{ s}$), 因此为 **memory-bound**. (此时 H100 的 accelerator intensity = 295.3731.)

Note 6 (Accelerator Intensity). 使用某硬件执行任务时可以定义该硬件的 accelerator intensity $K = \frac{\text{FLOP/s}}{\text{bytes/s}}$, 即每秒能够完成的计算量与能够处理的数据量的比值. 若 arithmetic intensity $>$ accelerator intensity, 即 $\frac{\text{FLOPs}}{\text{FLOP/s}} > \frac{\text{Bytes Moved}}{\text{bytes/s}}$, 则计算时间更长, 为 **compute-bound**; 反之为 **memory-bound**. H100 的 accelerator intensity $K = 295.3731$.

Example 3 (GeLU). 对于 GELU 激活函数 $GELU(x_i) = 0.5 \times x_i \times (1 + \tanh(\sqrt{\frac{2}{\pi}} \times (x_i + 0.044715 \times x_i^3)))$, $x \in \mathbb{R}^N$. 读取和写入均需 $2N$ bytes, 计算需要 $20N$ ($\tanh$ 需要逼近计算), 此时 $I_{\text{GELU}} = \frac{20N}{4N} = 5 < K$, 为 **memory-bound**.

Note 7. 虽然 ReLU 的计算量明显少于 GELU, 但由于他们都是 **memory-bound**, 都需要等待 memory 移动数据, 因此在 GPU 计算中 ReLU 并不比 GELU 更快.

Example 4 (Dot Product). 对于向量点积 $x @ y, x, y \in \mathbb{R}^N$, 读取 x, y 各需要 $2N$ bytes, 写入输出需要 2 bytes, 计算需要 $N + N - 1$ (N 乘法, $N - 1$ 加法) FLOPs, 此时 $I_{\text{vec.dot}} = \frac{2N-1}{4N+2} \approx 0.5 < K$, 因此为 **memory-bound**.

Example 5 (Matrix Vector Product). 对于向量与矩阵相乘 $W @ x, W \in \mathbb{R}^{N \times N}, x \in \mathbb{R}^N$, 读取输入需 $2 \times (N + N^2)$ bytes, 写入输出需 $2N$ bytes, 计算需 $N(N + N - 1)$ FLOPs, 此时 $I = \frac{N(2N-1)}{2N(1+N)} \approx 1 < K$, 因此为 **memory-bound**.

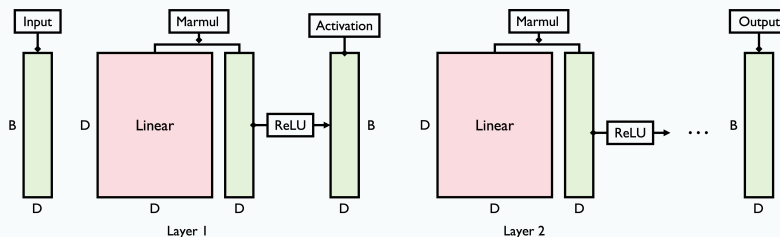
Example 6 (Matmul). 对于矩阵乘法 $WX, W, X \in \mathbb{R}^{N \times N}, N = 1024$, 读取输入需要 $4N^2$ bytes, 写入输出需要 $2N^2$ bytes, 计算需要 $N^2(2N - 1)$, 此时 $I_{\text{matmul}} = \frac{N^2(2N-1)}{6N^2} = \frac{2N-1}{6N} \approx 341.1667 > K$, 因此为 **computer-bound**.

Note 8. ① 只有当大型矩阵乘法计算占据主导时才会出现 **compute-bound**. ② 在大模型推理 (inference) 阶段主要进行的计算是矩阵向量乘积 (matrix-vector product), 因此 inference 阶段是 **memory-bound** 的. ③ Arithmetic Intensity 与 Accelerator Intensity 都与浮点数类型相关.

Memory Accounting for Real Training

Operation Accounting for Real Training

Deep Linear Network. 考虑一个简单的全连接神经网络 (MLP)，共 L 层，使用 ReLU 激活函数，输入维度 $B \times D$ ，线性矩阵维度 $D \times D$ ，浮点数类型为 BFloat16. 此时参数总量为 $D \times D \times L$.



One Layer Gradient. 若只考虑全连接网络中的一层 $h_1 \xrightarrow{w_2} h_2$, $B = 1024, D = 256$, 那么整个过程分为两部分: ① **Forward pass:** 只考虑 Matmul 过程, 计算量为 $B \times D \times (2D - 1) \approx 2BD^2 = 134,217,728$ FLOPs. ② **Backward pass:** (1) $h_1.grad = \frac{d \text{loss}}{dh_1} \in \mathbb{R}^{B \times D}$, 计算量为 $2BD^2$ FLOPs (2) $w_2.grad = \frac{d \text{loss}}{dw_2} \in \mathbb{R}^{D \times D}$, 计算量为 $2BD^2$ FLOPs. 因此反向传播计算量为 $4BD^2 = 268,435,456$ FLOPs. (Backward 的计算量是 forward 的两倍.)

All Layers Gradient. 当考虑整个 MLP 时, forward pass 需要 $2 \times \# \text{data points} \times \# \text{parameters}$ FLOPs, backward pass 需要 $4 \times \# \text{data points} \times \# \text{parameters}$ FLOPs, 整个计算过程需要 $6 \times \# \text{data points} \times \# \text{parameters}$ FLOPs.

Optimizer Step. Optimizer step 的计算量取决于模型参数数量和使用的优化器, 例如考虑 SGD $W \leftarrow W - \eta G$, 那么每一层需要 $2 \times D^2$ FLOPs (乘法、加法各 D^2 FLOPs), 整个模型的 optimizer step 需要 $2D^2 L$ FLOPs.

Memory Accounting for Real Training

Memory Accounting. 同样考虑一个简单的 MLP, 但使用混合精度, 此时各部分的 memory 量如下:

1. Parameter memory: $2 \times D^2 \times L$ bytes (BFloat16 占 2 bytes, 每层 D^2 参数, 共 L 层)
2. Activation memory: $2 \times BD \times L$ bytes (BFloat16 占 2 bytes, 每层 BD 参数, 共 L 层)
3. Gradient memory: $2 \times \text{parameter number}$ bytes (BFloat16 占 2 bytes, 梯度维度与参数维度相同)
4. Optimizer memory: $4 \times \text{parameter number}$ (Float32 占 4 bytes, SGD 只需要保存梯度)

Note 9. 为了使更新更稳定因此 optimizer step 使用 Float32. 若考虑 Adam 优化器, 由于存储 second momentum 也需 $4 \times \text{parameter number}$, 因此一共 $8 \times \text{parameter number}$.

Memory Optimization. 可以看到很多任务都受限于 memory (memory-bound), 常见的优化方式有:

1. **Gradient Accumulation:** 虽然较大的 batch size 可以提高梯度更新的稳定性 (使用的数据更多), 但 activation memory 与 batch size 正相关, 为了限制 activation memory 可以使用梯度累积, 即用较小的 micro-batch 模拟一较大的 effective batch, 每次 micro-batch 计算完即可清空对应 memory 继续计算.
2. **Activation Checkpointing:** 也称为 gradient checkpointing 和 rematerialization, 由于训练时总 memory 与总层数 L 相关, 因此可以考虑 Forward 时只保存少数检查点, 丢弃检查点之间的中间 activations, Backward 时再重新计算它们.

Summary

Summary

① 任何操作都以 tensor 为基础. ② Einops 可以让 tensor operation 更易理解. ③ 每一 training step 的计算量约为:

$$6 \times \# \text{ data points} \times \# \text{ parameters FLOPs.} \quad (5)$$

④ 大多数基本操作都是 memory-bound, 唯有大规模矩阵乘法是 compute-bound. ⑤ Reduce memory 的方法有: (1) Gradient Accumulation (2) Activation Checkpointing.

Last updated: August 13, 2026

References